
S/W Detailed Level Design

Project Name	Currency Exchange System — Release 4		
Block Name	Compile Commander		
Author	Magnus Jaaska	Approver	
Team	Compile Commander		

S/W Detailed Level Design

This document represents Detailed Level Design (DLD). It describes the detailed system design and implementation plan in alignment with Agile principles. The DLD is updated incrementally with each release to reflect system evolution.

Contents

1.	Overview.....	4
2.	System Overview / Architectural Context.....	5
3.	UML Class Diagram (Technical Design)	6
4.	Class Specifications	7
5.	Interfaces and Abstractions	8
6.	Function Responsibilities	8
7.	Operation Flow	10
8.	Enumerations & Constants	11
9.	Validation Rules & Future Work	12
10.	Traceability Matrix.....	12
11.	Code Structure and File Mapping	13
12.	Revision History	14
	Release 3.....	15
13.	Validation Rules & Preconditions/Postconditions	15
14.	Behavioral Models	17
15.	Error & Exception Handling Policy	18
16.	Revision History.....	20

■ Revision History

Version	Date	Revised contents	Author	Approver
2.0	28.10.2025	Initial Release 2 DLD; three-layer refactor; interfaces; method responsibilities; traceability.	Magnus Jaaska	
4.0	19.12.2025	Initial Release 4 DLD; file persistence; loadAll/saveAll methods; file format specification; validation rules	Magnus Jaaska	

■ Terms and Abbreviations

Term	Description
UI	User Interface (ConsoleUI)
DLD	Detailed Level Design document
SRS	Software Requirements Specification
DIP	Dependency Inversion Principle
SRP	Single Responsibility Principle
I/O	Input/Output (file operations)

S/W Detailed Level Design

■ References

1. SW Requirements Specification (docs/release-1/SRS.pdf)
2. Lecture 5–7 (Design for Quality, Modularity, Abstraction, Polymorphism)
3. Example DLD Currency Exchange (reference structure)
4. Student Guide DLD R4 (release4_guide/Student_Guide_DLD_R4.pdf)

1. Overview

The Currency Exchange System automates currency conversion, verifies reserves before payout, records transactions in memory, and provides a daily summary. Release 4 extends the system with persistent file storage. This DLD defines the Release 4 design: file I/O methods (loadAll, saveAll), file format specification, parsing/validation logic, lifecycle management (startup/shutdown), and integration with the existing three-layer architecture (UI → Logic → Data). The system now supports complete data persistence across program executions.

2. System Overview / Architectural Context

Release 4 maintains the three-layer architecture established in Releases 2 and 3, adding a Persistence Layer.

- **UI:** ConsoleUI — console interaction & formatting only.
- **Logic:** ExchangeManager — coordinates conversion, reserve checks, and reporting.
- **Data:** ICurrencyRepository / CurrencyRepository — in-memory balances & exchange rates + file I/O.
- **Persistence:** data.txt - text file for permanent storage of currency balances

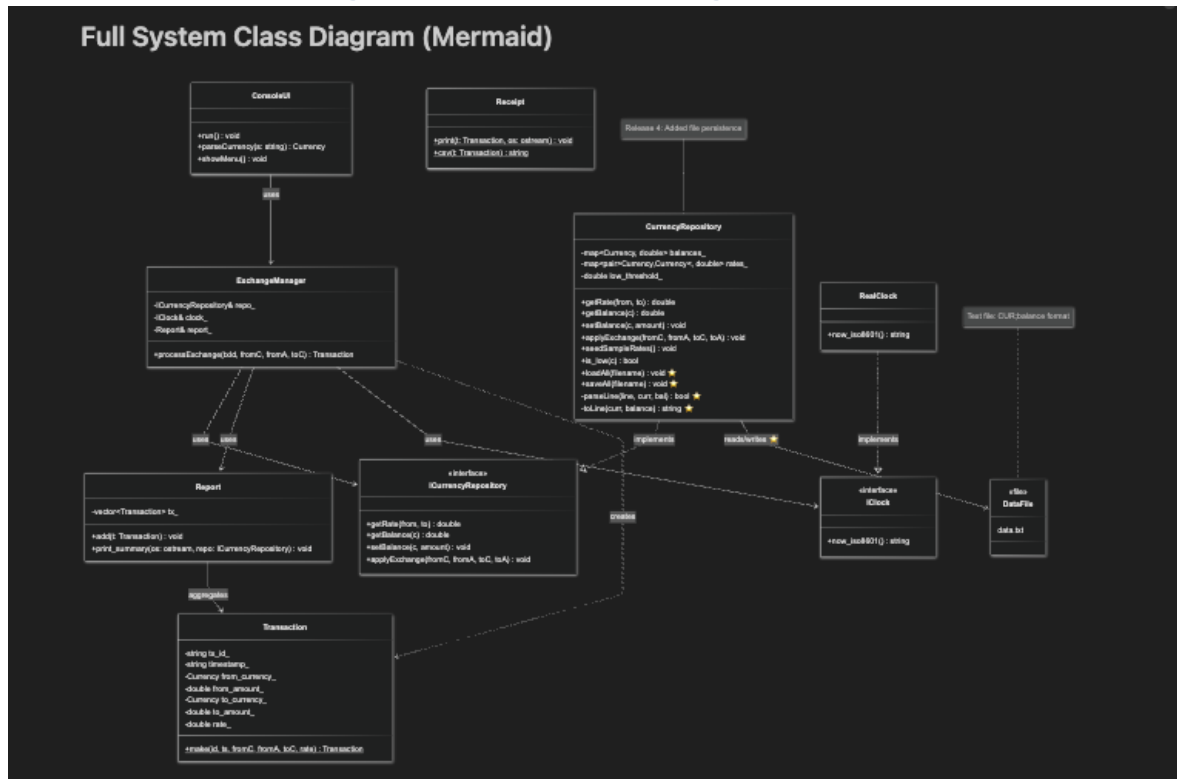
Principles: Separation of Concerns, SRP, DIP, encapsulation via header/source split, RAII for file handling.

Data Flow:

- **Startup:** File → loadAll() → Repository (in-memory)
- **Runtime:** UI → Logic → Repository (fast in-memory operations)
- **Shutdown:** Repository → `saveAll()` → File (persistent storage)

S/W Detailed Level Design

3. UML Class Diagram (Technical Design)



Note: All other classes (ConsoleUI, ExchangeManager, Transaction, Report, Receipt, IClock, RealClock) remain unchanged from Release 3. Indicates Release 4 additions.

4. Class Specifications

Class	Type	Description	Attributes	Methods
ConsoleUI	UI	Console I/O and formatting; no business logic.		run(), parseCurrency(s), showMenu()
ExchangeManager	Logic	Coordinates exchange; checks reserve; updates report.	repo_: ICurrencyRepository& clock_: IClock& report_: Report&	processExchange(txId, fromC, fromA, toC): Transaction
ICurrencyRepository	Interface	Abstracts data access for rates/balances.		getRate(from,to), getBalance(c), setBalance(c,amt), applyExchange(fromC,fromA,toC,toA)
CurrencyRepository	Data	In-memory balances & rates; implements interface.	balances_, rates_, low_threshold_	getRate(), getBalance(), setBalance(), applyExchange(), is_low(c), seedSampleRates()
Transaction	Domain	Immutable exchange record w/ computed to-amount.	tx_id_, timestamp_, from_currency_, from_amount_, to_currency_, to_amount_, rate_	static make(id,ts,fromC,fromA,toC,rate): Transaction
Report	Domain	Aggregates transactions; prints daily summary.	tx_: vector<Transaction>	add(t), print_summary(os, repo)
Receipt	Utility	Formats receipt and CSV line.		print(t, os), csv(t): string
IClock	Interface	Time abstraction.		now_iso8601(): string

S/W Detailed Level Design

RealClock	Service	System clock implementation.	now_iso8601(): string
CurrencyRepository	Data	In-memory balances & rates; implements file I/O	balances_, rates_, low_threshold_ [Existing] getRate(), getBalance(), setBalance(), applyExchange(), is_low(c), seedSampleRates() [NEW] loadAll(filename), saveAll(filename), parseLine() [private], toLine() [private]

5. Interfaces and Abstractions

Interface	Purpose	Key Methods	Planned For (Release)
ICurrencyRepository	Decouple logic from storage; enable File/DB later	getRate(), getBalance(), setBalance(), applyExchange()	R2 (implemented)
IClock	Decouple time for tests	now_iso8601()	R2 (implemented)

Release 4 Note: File I/O operations are implementation details of CurrencyRepository, not exposed through the interface. This maintains clean separation: Logic layer remains unaware of persistence mechanism.

6. Function Responsibilities

Class	Method	Purpose	Input	Output	Notes
CurrencyRepository	loadAll()	Load balances from file into memory	filename (string)	void	Skips invalid lines; never throws for malformed data
CurrencyRepository	saveAll()	Save all balances from	filename (string)	Void	Throws exception if file cannot be opened

TITLE S/W Detailed Level Design

		memory to file			
CurrencyRepository	parseLine()	Parse one file line into Currency and balance	line (string), curr (out), bal (out)	bool	Returns true if valid, false otherwise
CurrencyRepository	toLine()	Format one balance entry into file line curr	curr (Currency), balance (double)	string	Format one balance entry into file line curr (Currency), balance (double) string Produces "CUR;balance" format
ExchangeManager	processExchange()	Computes payout, checks reserve, applies exchange, records transaction	txId, fromC, fromA, toC	Transaction	Throws on insufficient reserve/bad rate
CurrencyRepository	applyExchange()	Update balances for from/to	fromC, fromA, toC, toA		Encapsulated state change
Report	add()	Append to daily collection	Transaction		Called after success

S/W Detailed Level Design

Report	print_summary()	Emit daily totals/closing balances	ostream, ICurrencyRepository		Presentation kept minimal
Receipt	print()	Human- readable receipt	Transaction, ostream		UI use
Receipt	csv()	CSV transaction line	Transaction	string	For logging later
RealClock	now_iso8601()	Timestamp		string	Local ISO- 8601
ConsoleUI	run()	Interactive loop; parse/execute	stdin	stdout	Uses manager; handles errors

7. Operation Flow

Startup Flow (Release 4)

1. main() creates CurrencyRepository.
2. main() calls repo.loadAll("data.txt").
3. loadAll() opens file for reading.
4. If file doesn't exist → empty repository (valid state).
5. If file exists → read line by line.
6. For each line: call parseLine(line, curr, balance).
7. Valid line → update balances_[currency] = balance.
8. Invalid line → log warning, skip.
9. Report loaded count.
10. If balances empty → initialize with defaults.
11. Continue to UI interactive session.

Runtime Flow (Unchanged from Release 3)

1. ConsoleUI reads input ("EUR 100 USD").
2. Parses → ExchangeManager::processExchange().
3. Manager gets rate from ICurrencyRepository, computes payout.
4. Validates reserves → applies exchange in repo.
5. Creates Transaction, adds to Report.
6. UI prints Receipt; session continues.

Shutdown Flow (Release 4)

1. User enters "quit".
2. UI prints end-of-day report (Report::print_summary()).
3. main() calls repo.saveAll("data.txt").
4. saveAll() opens file for writing (overwrites existing).
5. For each (currency, balance) in balances_:
6. Call toLine(currency, balance) → formatted string.
7. Write line to file.
8. Close file.
9. Report saved count.
10. Program terminates.

8. Enumerations & Constants

Name	Value / Type	Description
enum class Currency	{EUR, USD, GBP, SEK, DKK}	Supported currencies
LOW_THRESHOLD	double = 100.0	Alert threshold for low balances
INITIAL_BALANCES	per-currency doubles	Startup config for interactive run
DATA_FILENAME	"data.txt"	Default persistent storage file

S/W Detailed Level Design**9. Validation Rules & Future Work**

Rule / Planned Feature	Description	Target
Input validation & errors	amount > 0, valid currency codes; clear error messages	R3
Strict reserve enforcement	Block payout > reserve; error codes	R3
Exception policy	try/catch at UI; noexcept in Data	R3
File I/O persistence	Save/load rates, tx, reports	R3
Unit tests	Mock IClock & repo; per-layer tests	R3
File I/O persistence	Save/load balances from data.txt	R4
File format validation	Validate currency codes, numeric balances, line format	R4
Malformed data handling	Skip invalid lines with warnings	R4

10. Traceability Matrix

Requirement (SRS)	Class / Method (DLD)	Notes
RQ-01 Convert currency	ExchangeManager::processExchange()	Core conversion via repo rate
RQ-02 Show receipt	Receipt::print(), ConsoleUI::run()	UI formatting only
RQ-03 Maintain reserves	CurrencyRepository::applyExchange()	Encapsulated data update
RQ-04 Daily report	Report::add(), Report::print_summary()	End-of-day summary
RQ-05 Time stamps	RealClock::now_iso8601()	Abstracted time
RQ-06 Persistent storage	CurrencyRepository::loadAll(), saveAll()	File-to-memory lifecycle
RQ-07 Data preservation	CurrencyRepository::parseLine(), toLine()	File format encapsulation
RQ-08 Malformed data	CurrencyRepository::parseLine() validation	Skip invalid lines, continue

11. Code Structure and File Mapping

Class / Module	Files
ConsoleUI	include/console_ui.h, src/console_ui.cpp
ExchangeManager	include/exchange_manager.h, src/exchange_manager.cpp
ICurrencyRepository	include/currency_repository.h (interface)
CurrencyRepository	include/currency_repository.h, src/currency_repository.cpp
Transaction	include/transaction.h, src/transaction.cpp
Report	include/report.h, src/report.cpp
Receipt	include/receipt.h, src/receipt.cpp
IClock / RealClock	include/iclock.h, src/iclock.cpp
Entry point	src/main.cpp
Persistent Storage	data.txt [NEW]
Build	Makefile

12. Revision History

Date	Version	Change Summary	Author
28.10.2025	1.0	Initial DLD created from Release 2 refactor	Magnus Jaaska
19.12.2025	4.0	Initial Release 4 DLD; file persistence; loadAll/saveAll methods; file format specification; validation rules	Magnus Jaaska

Release 3

13. Validation Rules & Preconditions/Postconditions

< Describe how your system checks input data and enforces correctness before and after operations. Each method should list clear preconditions (what must be true before it runs) and postconditions (what must be true after it completes). Indicate the layer where validation happens - UI, Logic, or Repository. Use the same class and method names as in Release 2.>

Class	Method	Preconditions	Postconditions	Validation Level (UI/Logic/Repository)	Explanation
CurrencyRepository	loadAll()	filename is valid string; Repository initialized	balances_ contains all valid entries from file; invalid lines skipped; no exceptions for malformed data	Repository	Validates file format, currency codes, balance values; defensive parsing
CurrencyRepository	saveAll()	filename is valid string; Repository initialized; balances_ may be empty	File contains all entries from balances_; previous content overwritten; throws if write fails	Repository	Critical operation; must succeed or throw exception
CurrencyRepository	ParseLine()	line is valid string; curr and balance are valid references	Returns true if valid (curr and balance set); returns false if invalid (parameters unchanged); never throws	Repository	Strict format validation; single responsibility
CurrencyRepository	toLine()	curr is valid Currency enum; balance is valid double	Returns formatted string matching file specification; always produces valid format	Repository	Assumes inputs valid (internal method); simple formatting
ConsoleUI	run()	Input fields present	Input values captured	UI	Validates all required fields are entered
ExchangeManager	processExchange()	fromA >= 0, currency codes valid	Transaction created, balances updated	Logic	Validates business rules before

Name

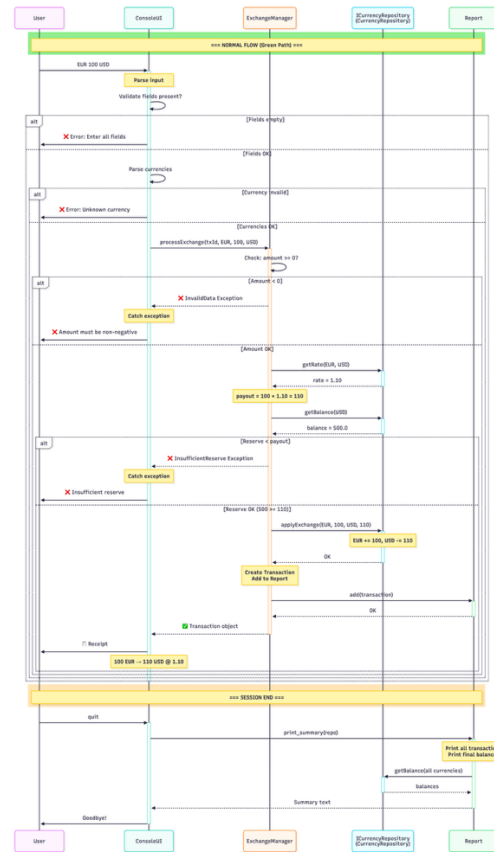
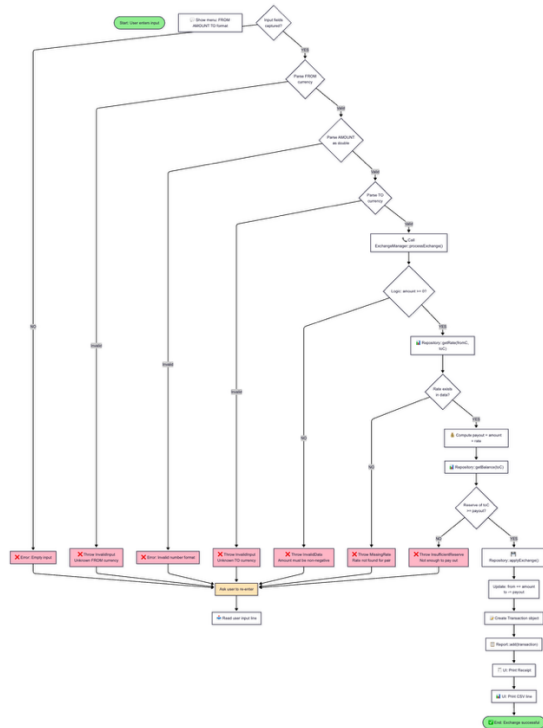
Date

S/W Detailed Level Design

					Repository call
CurrencyRepository	getRate	Rate pair exists	Returns rate or throws exception	Repository	Confirms data availability
CurrencyRepository	applyExchange	All validations passed	Balances updated successfully	Repository	Applies validated exchange

14. Behavioral Models

< Attach two UML diagrams that describe one key operation in your system, for example, processing an order, confirming a booking, or updating a record. Both diagrams must describe the same operation to show consistency.



S/W Detailed Level Design

15. Error & Exception Handling Policy

< List all exceptions that may occur in your system. Describe where each one is thrown, where it is caught, what message it shows, and what the program does afterwards.

For this release, use only the three architectural layers: UI, Logic, Repository. >

Exception Type	Thrown By (Layer / Class)	Caught At (Layer)	Message / what()	Default Action (message / stop / retry)
FileReadError	Repository — CurrencyRepository::loadAll()	UI (main.cpp)	"Warning: Could not open data file {filename}. Starting with empty balances."	Show message → continue with defaults
FileWriteError	Repository — CurrencyRepository::saveAll()	UI (main.cpp)	"ERROR: Could not save data to {filename}. Data may be lost."	Show message → exit (critical failure)
N/A (skip line)	Repository — CurrencyRepository::parseLine()	Internal (loadAll)	"Warning: Skipping invalid line {lineNumber}: {content}"	Log warning → continue loading other lines
InvalidInput	UI — ConsoleUI input parsing or parseCurrency()	UI	"Please enter all required fields."	Show message → ask user to re- enter
InvalidInput	UI — ConsoleUI parseCurrency()	UI	"Unknown currency: {code}. Supported: EUR, USD, GBP, SEK, DKK"	Show message → ask user to re- enter
InvalidData	Logic — ExchangeManager::processExchange() guard clause	UI	"Amount must be non- negative."	Show message → stop operation

InsufficientReserve	Logic — ExchangeManager::processExchange() reserve check	UI	"Insufficient reserve of {currency} to pay out {amount}."	Show message → stop operation
MissingRate	Repository — CurrencyRepository::getRate()	UI	"Exchange rate not found for pair {from}/{to}."	Show message → stop operation

S/W Detailed Level Design

16. Revision History

<Track document changes across releases. >

Date	Version	Change Summary	Author
28.10.2025	1.0	Initial Release 2 DLD; three-layer refactor; interfaces; method responsibilities; traceability.	Magnus Jaaska
23.11.2025	2.0	Release 3 update: Added Section 13 (Validation Rules), Section 14 (Activity & Sequence Diagrams), Section 15 (Exception Policy). Documented processExchange() as core operation with complete validation, behavior, and error handling flow.	Magnus Jaaska
19.12.2025	3.0	Release 4 update: File persistence (loadAll/saveAll), file format specification, Mermaid diagrams for classes and file operations	Magnus Jaaska